



Travaux Pratiques
Développement Mobile avec
Flutter

Master 1 Informatique · Année universitaire 2024–2025
10 TP progressifs · Mini-projet intégrateur · Examen pratique

De zéro à une application mobile professionnelle

Prérequis & Configuration de l'environnement

Avant de commencer les travaux pratiques, vous devez configurer votre environnement de développement Flutter. Suivez attentivement les étapes ci-dessous.

1. Logiciels requis

Logiciel	Version	Rôle	Lien
Flutter SDK	≥ 3.19	Framework principal	flutter.dev
Dart SDK	Inclus	Langage de programmation	dart.dev
Android Studio	≥ Hedgehog	IDE + Émulateur Android	developer.android.com
VS Code	≥ 1.88	Éditeur alternatif	code.visualstudio.com
Git	≥ 2.40	Gestion de version	git-scm.com
Java JDK	17 (LTS)	Requis pour Android	adoptium.net

2. Installation de Flutter

Étape 1 — Téléchargement

Rendez-vous sur flutter.dev/docs/get-started/install et choisissez votre système d'exploitation (Windows, macOS ou Linux). Téléchargez l'archive correspondante.

Étape 2 — Extraction et PATH

Extrayez l'archive dans un répertoire sans espaces ni caractères spéciaux (ex : C:\src\flutter sur Windows ou ~/development/flutter sur macOS/Linux).

Ajoutez le répertoire bin de Flutter à votre variable d'environnement PATH :

```
# macOS / Linux – ajouter dans ~/.zshrc ou ~/.bashrc
export PATH="$PATH:~/development/flutter/bin"

# Windows – via Paramètres système > Variables d'environnement
# Ajouter : C:\src\flutter\bin
```

Étape 3 — Vérification avec flutter doctor

Ouvrez un terminal et exécutez :

```
flutter doctor

# Exemple de sortie attendue :
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.19.x, on macOS 14.x)
[✓] Android toolchain - develop for Android devices
[✓] Android Studio (version Hedgehog)
[✓] VS Code (version 1.88.x)
```

```
[✓] Connected device (2 available)
[✓] Network resources
```

3. Configuration d'Android Studio

Ouvrez Android Studio et suivez ces étapes :

1. Installez les plugins Flutter et Dart : File > Settings > Plug-ins > recherchez 'Flutter' > Install
2. Créez un émulateur : Device Manager > Create Device > Pixel 7 Pro > Android 14 (API 34) > Finish
3. Installez les Android SDK tools : SDK Manager > SDK Tools > cochez 'Android SDK Command-line Tools'
4. Acceptez les licences Android : exécutez flutter doctor --android-licenses dans le terminal

4. Extensions VS Code recommandées

Extension	Éditeur	Utilité
Flutter	Dart Code	Support Flutter complet (autocomplétion, debug)
Dart	Dart Code	Support du langage Dart
Pubspec Assist	Jeroen Meijer	Ajouter des dépendances facilement
Error Lens	Alexander	Afficher les erreurs inline
Bracket Pair Colorizer	CoenraadS	Colorer les accolades imbriquées
GitLens	GitKraken	Visualiser l'historique Git

5. Commandes Flutter essentielles

Commande	Description	Usage
flutter create nom_app	Crée un nouveau projet Flutter	Début de chaque TP
flutter run	Lance l'app sur l'émulateur/appareil	Développement
flutter run -d chrome	Lance l'app dans Chrome	Tests web
flutter hot reload (r)	Recharge l'UI sans redémarrer	Développement rapide
flutter hot restart (R)	Redémarre l'app complètement	Reset de l'état
flutter pub add package	Ajoute une dépendance	Intégration librairies
flutter pub get	Télécharge les dépendances	Après modif pubspec

flutter build apk	Génère l'APK Android	Production
flutter analyze	Analyse statique du code	Qualité du code
flutter test	Lance les tests unitaires	Tests automatisés
flutter clean	Nettoie le build cache	Résolution de bugs

6. Lancer un émulateur

```
# Lister les émulateurs disponibles
flutter emulators

# Lancer un émulateur spécifique
flutter emulators --launch Pixel_7_Pro_API_34

# Vérifier les appareils connectés
flutter devices

# Lancer l'app sur tous les appareils connectés
flutter run -d all
```

Conseil

Si flutter doctor affiche des avertissements en jaune ([!]), lisez attentivement le message — il vous indique généralement la commande exacte à exécuter pour corriger le problème. Les erreurs en rouge ([X]) doivent être résolues avant de commencer.

TP 1

Hello Flutter — Découverte de l'environnement

Durée : 2h · Niveau : Débutant absolu

Objectifs pédagogiques

- 1 Créer et exécuter son premier projet Flutter
- 2 Comprendre la structure d'un projet Flutter
- 3 Maîtriser les widgets de base : Text, Container, Column, Row, Image
- 4 Distinguer StatelessWidget et StatefulWidget
- 5 Personnaliser le thème global d'une application
- 6 Utiliser Hot Reload pour un développement efficace

Contexte du projet

Scénario

Une agence de communication souhaite créer une application de carte de visite numérique interactive pour ses consultants. L'application doit afficher les informations du consultant (nom, poste, contact), son avatar, ses compétences et ses réseaux sociaux sous forme de boutons. Ce TP vous permettra de réaliser la version statique de cette carte de visite.

Consignes détaillées

Étape 1 — Création du projet

Ouvrez un terminal dans le répertoire de votre choix et créez le projet :

```
flutter create carte_visite
cd carte_visite
flutter run
```

Vérifiez que l'application de démonstration Flutter (le compteur) se lance correctement sur votre émulateur.

Étape 2 — Nettoyage du projet

Ouvrez lib/main.dart. Supprimez tout le contenu et remplacez-le par une structure minimale :

```
import 'package:flutter/material.dart';

void main() => runApp(const CarteVisiteApp());
```

```
class CarteVisiteApp extends StatelessWidget {
  const CarteVisiteApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Carte de Visite',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.indigo),
        useMaterial3: true,
      ),
      home: const CarteVisiteScreen(),
    );
  }
}
```

Étape 3 — Structure des fichiers

Créez l'arborescence suivante dans le projet :

```
lib/
├── main.dart           ← Point d'entrée
├── screens/
│   └── carte_visite_screen.dart
├── widgets/
│   ├── avatar_widget.dart
│   ├── info_card.dart
│   └── competence_chip.dart
└── utils/
    └── constants.dart ← Couleurs, styles
```

Étape 4 — Créer l'écran principal

Dans lib/screens/carte_visite_screen.dart, créez le widget principal :

```
class CarteVisiteScreen extends StatelessWidget {
  const CarteVisiteScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Container(
        decoration: BoxDecoration(
          gradient: LinearGradient(
            begin: Alignment.topLeft,
            end: Alignment.bottomRight,
            colors: [Colors.indigo.shade800, Colors.blue.shade400],
          ),
        ),
      ),
      child: SafeArea(
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(24),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              SizedBox(height: 40),
              // AVATAR
              CircleAvatar(
                radius: 60,
                backgroundImage: NetworkImage(
                  'https://i.pravatar.cc/200',
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

```

const SizedBox(height: 20),
// NOM
Text('Aminata Diallo',
  style: Theme.of(context).textTheme.headlineMedium?.copyWith(
    color: Colors.white, fontWeight: FontWeight.bold)),
// POSTE
Text('Développeuse Mobile Senior',
  style: TextStyle(color: Colors.white70, fontSize: 16)),
// ... Reste du contenu
],
),
),
),
),
);
}

```

Étape 5 — Ajouter les sections

Ajoutez les éléments suivants dans la Column, dans cet ordre :

- Section Contact : Row avec icônes (email, téléphone, localisation) + textes
- Divider avec texte 'Compétences'
- Wrap de Chips pour les compétences techniques
- Divider avec texte 'Réseaux sociaux'
- Row avec boutons circulaires pour LinkedIn, GitHub, Twitter

Étape 6 — StatefulWidget : compteur de vues

Transformez CarteVisiteScreen en StatefulWidget. Ajoutez un compteur de 'vues du profil' qui s'incrémente à chaque appui sur un bouton ElevatedButton. Affichez ce compteur en haut de l'écran.

Étape 7 — Personnalisation

Remplacez les données fictives par vos propres informations (nom, compétences, etc.) et ajustez le schéma de couleurs selon vos préférences.



Interface attendue

Zone	Composant Flutter	Description visuelle
En-tête	Container + LinearGradient	Fond dégradé indigo → bleu
Avatar	CircleAvatar + NetworkImage	Photo ronde avec bordure blanche
Identité	Column + Text	Nom en grand, poste en gris clair
Contact	Row + Icon + Text	Email, téléphone, ville sur une ligne
Compétences	Wrap + Chip	Tags colorés (Flutter, Dart, Firebase...)
Réseaux sociaux	Row + IconButton	Boutons circulaires avec icônes

Compteur	Text + ElevatedButton	Nombre de vues + bouton '+1'
----------	-----------------------	------------------------------

Arborescence du projet

```

carte_visite/
├── lib/
│   ├── main.dart
│   ├── screens/
│   │   └── carte_visite_screen.dart
│   ├── widgets/
│   │   ├── avatar_widget.dart
│   │   ├── contact_row.dart
│   │   └── competence_chip.dart
│   └── utils/
│       └── app_theme.dart
├── assets/
│   └── images/
├── pubspec.yaml
└── README.md

```

Concepts Flutter utilisés

StatelessWidget	StatefulWidget	MaterialApp
Scaffold	Container	Column
Row	Text	Icon
CircleAvatar	Chip	Wrap
ElevatedButton	IconButton	LinearGradient
SafeArea	SingleChildScrollView	SizedBox
Padding	setState()	ThemeData

Bonus & défis

- Ajouter une animation d'entrée sur l'avatar (AnimatedContainer)
- Implémenter un bouton de bascule thème clair / sombre
- Ajouter un bouton 'Partager' qui ouvre le partage natif du téléphone
- Rendre l'image de profil cliquable pour l'agrandir (Dialog)
- Ajouter une section 'Expériences' avec un ListView horizontal

Critères d'évaluation

Critère	Points	Description
---------	--------	-------------

Application fonctionnelle	20 pts	L'app compile et s'exécute sans erreur
Structure du code	15 pts	Séparation screens/widgets/utils respectée
Widgets requis	25 pts	Tous les widgets demandés sont présents
StatefulWidget	20 pts	Compteur de vues fonctionnel avec setState()
UI/UX	10 pts	Interface propre, lisible, design soigné
Personnalisation	10 pts	Données remplacées par les infos de l'étudiant
TOTAL	100 pts	

TP 2

Application Todo — Listes & Gestion d'état de base

Durée : 3h · Niveau : Débutant

Objectifs pédagogiques

- 1 Maîtriser ListView et ses variantes (ListView.builder, ListView.separated)
- 2 Gérer l'état local avec setState() dans un StatefulWidget
- 3 Créer et utiliser des modèles de données (classes Dart)
- 4 Utiliser les formulaires Flutter : TextField, TextEditingController
- 5 Implémenter les actions utilisateur : ajout, suppression, modification
- 6 Utiliser Dismissible pour les gestes de balayage

Contexte du projet

Scénario

Une startup EdTech souhaite créer une application de gestion de tâches pour ses étudiants. L'application doit permettre de créer des tâches avec titre, description et priorité, de les marquer comme terminées, de les filtrer par statut et de les supprimer par balayage. Vous allez réaliser la version MVP (Minimum Viable Product) de cette application.

Consignes détaillées

Étape 1 — Modèle de données

Créez lib/models/todo.dart avec une classe Todo :

```
class Todo {  
  final String id;  
  String title;  
  String description;  
  bool isCompleted;  
  TodoPriority priority;  
  final DateTime createdAt;  
  
  Todo({  
    required this.title,  
    this.description = '',  
    this.isCompleted = false,  
    this.priority = TodoPriority.medium,  
  }) : id = DateTime.now().millisecondsSinceEpoch.toString(),
```

```

        createdAt = DateTime.now();
    }
    enum TodoPriority { low, medium, high }

```

Étape 2 — Écran principal avec ListView.builder

L'écran principal doit afficher la liste des todos avec `ListView.builder`. Chaque item doit montrer : checkbox, titre, priorité (badge coloré), et bouton suppression.

- Checkbox pour cocher/décocher une tâche
- Titre barré si la tâche est complétée (`TextDecoration.lineThrough`)
- Badge de priorité coloré (vert/orange/rouge)
- Balayage (`Dismissible`) pour supprimer

Étape 3 — Formulaire d'ajout

Créez un Bottom Sheet (`showModalBottomSheet`) contenant :

- `TextField` pour le titre (obligatoire)
- `TextField` multiline pour la description
- `SegmentedButton` ou `RadioListTile` pour la priorité
- Bouton 'Ajouter' qui valide et ferme le sheet

Étape 4 — Filtres

Ajoutez une `TabBar` ou des `FilterChip` en haut de l'écran pour filtrer :

- Toutes les tâches
- Tâches en cours
- Tâches complétées

Étape 5 — Statistiques

Ajoutez un bandeau en bas ou en haut affichant : total des tâches, nombre complétées, pourcentage de progression avec une `LinearProgressIndicator`.



Interface attendue

Écran/Zone	Description
AppBar	Titre 'Mes Tâches', icône filtre, compteur actif
Bandeau stats	Card avec progression, stats complétées/total
Filtre	FilterChip ou TabBar (Toutes / En cours / Terminées)
ListView	Items avec checkbox, titre, badge priorité, Dismissible
FAB	FloatingActionButton '+' pour ajouter une tâche
Bottom Sheet	Formulaire d'ajout avec validation

Empty state	Illustration + texte si aucune tâche
-------------	--------------------------------------

Arborescence du projet

```

todo_app/
├── lib/
│   ├── main.dart
│   ├── models/
│   │   └── todo.dart
│   ├── screens/
│   │   └── todo_list_screen.dart
│   ├── widgets/
│   │   ├── todo_item.dart
│   │   ├── add_todo_sheet.dart
│   │   ├── priority_badge.dart
│   │   └── stats_card.dart
│   └── utils/
│       └── constants.dart
└── pubspec.yaml

```

Concepts Flutter utilisés

ListView.builder	ListView.separated	Dismissible
StatefulWidget	setState()	TextEditingController
ModalBottomSheet	TextField	Checkbox
FilterChip	LinearProgressIndicator	SnackBar
FloatingActionButton	AlertDialog	TabBar
Classe Dart	Enum	Liste<T>

Bonus & défis

- Ajouter une date d'échéance avec DatePicker
- Implémenter la modification d'une tâche existante
- Ajouter un tri par priorité ou par date
- Afficher une notification locale quand une tâche est terminée
- Animations de suppression et d'ajout (AnimatedList)

Critères d'évaluation

Critère	Points	Description
Modèle de données	15 pts	Classe Todo avec tous les attributs requis

ListView.builder	20 pts	Liste fonctionnelle avec items correctement affichés
Ajout de tâches	20 pts	Formulaire avec validation et Bottom Sheet
Suppression Dismissible	15 pts	Balayage pour supprimer avec confirmation
Filtres	15 pts	Filtrage par statut fonctionnel
Statistiques	10 pts	Barre de progression et compteurs
UI/UX	5 pts	Design cohérent et état vide géré
TOTAL	100 pts	

TP 3

Navigation & Architecture Multi-Écrans

Durée : 3h · Niveau : Intermédiaire débutant

Objectifs pédagogiques

- 1 Maîtriser la navigation impérative avec `Navigator.push/pop`
- 2 Passer des données entre écrans (arguments de route)
- 3 Implémenter une navigation avec `BottomNavigationBar`
- 4 Utiliser les routes nommées et `go_router`
- 5 Gérer le retour arrière et les résultats de navigation
- 6 Créer des transitions d'écran personnalisées

Contexte du projet

Scénario

Une application de gestion d'une bibliothèque universitaire doit permettre à l'utilisateur de naviguer entre le catalogue de livres, le détail d'un livre, les favoris et le profil utilisateur. Vous allez implémenter la navigation complète de cette application avec 4 onglets et des écrans de détail.

Consignes détaillées

Étape 1 — Modèle Livre

```
class Livre {  
  final String id, titre, auteur, genre, description, couverture;  
  final double note;  
  final int annee, pages;  
  bool isFavori;  
  Livre({required this.id, required this.titre, required this.auteur,  
    required this.genre, required this.description, required this.couverture,  
    required this.note, required this.annee, required this.pages,  
    this.isFavori = false});  
}
```

Étape 2 — `BottomNavigationBar`

Créez un écran racine `MainScreen` avec une `BottomNavigationBar` à 4 onglets :

- 🏠 Accueil — catalogue avec grille de livres
- 🔍 Recherche — barre de recherche + résultats
- ❤️ Favoris — liste des livres favoris
- 👤 Profil — informations de l'utilisateur

Étape 3 — Navigation vers le détail

Sur l'écran Catalogue, afficher les livres dans un `GridView.builder` (2 colonnes). Au tap sur un livre, naviguer vers l'écran de détail en passant l'objet `Livre` :

```
// Navigation avec données
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (_) => DetailLivreScreen(livre: livre),
  ),
);

// Récupérer un résultat au retour
final resultat = await Navigator.push<bool>(
  context,
  MaterialPageRoute(builder: (_) => DetailLivreScreen(livre: livre)),
);
if (resultat == true) setState(() { /* Mise à jour */ });
```

Étape 4 — Routes nommées

Configurez des routes nommées dans `MaterialApp` :

```
routes: {
  '/': (context) => const MainScreen(),
  '/livre/detail': (context) => const DetailLivreScreen(),
  '/profil/edit': (context) => const EditProfilScreen(),
},

// Navigation avec arguments
Navigator.pushNamed(context, '/livre/detail',
  arguments: {'livre': monLivre});

// Récupérer dans l'écran destination
final args = ModalRoute.of(context)!.settings.arguments
  as Map<String, dynamic>;
```

Étape 5 — Transitions personnalisées

Créez une transition personnalisée pour l'écran de détail (slide depuis le bas + fondu) :

```
Navigator.push(
  context,
  PageRouteBuilder(
    pageBuilder: (_, animation, __) => DetailLivreScreen(livre: livre),
    transitionsBuilder: (_, animation, __, child) {
      return SlideTransition(
        position: Tween<Offset>(
          begin: const Offset(0, 1),
          end: Offset.zero,
        ).animate(CurvedAnimation(parent: animation, curve: Curves.easeOut)),
        child: FadeTransition(opacity: animation, child: child),
      );
    },
  ),
);
```



Interface attendue

Écran	Composants principaux	Navigation
Catalogue	GridView.builder, BookCard	→ Détail au tap
Recherche	SearchBar, ListView, Chip genres	→ Détail au tap
Favoris	ListView, empty state	→ Détail au tap
Profil	CircleAvatar, ListTile, Switch	→ Édition profil
Détail livre	Hero image, rating, description, FAB favori	← Retour + résultat



Arborescence du projet

```

bibliotheque_app/
├── lib/
│   ├── main.dart
│   ├── models/
│   │   └── livre.dart
│   ├── screens/
│   │   ├── main_screen.dart      ← BottomNav racine
│   │   ├── catalogue_screen.dart
│   │   ├── recherche_screen.dart
│   │   ├── favoris_screen.dart
│   │   ├── profil_screen.dart
│   │   └── detail_livre_screen.dart
│   ├── widgets/
│   │   ├── book_card.dart
│   │   └── rating_stars.dart
│   └── data/
│       └── livres_data.dart      ← Données statiques
└── pubspec.yaml

```



Concepts Flutter utilisés

Navigator.push	Navigator.pop	MaterialPageRoute
BottomNavigationBar	GridView.builder	Hero
Routes nommées	PageRouteBuilder	SlideTransition
FadeTransition	SearchBar	IndexedStack
WillPopScope	arguments	Résultat de nav.

★ Bonus & défis

- Implémenter `go_router` pour une navigation déclarative moderne
- Ajouter une animation Hero sur l'image de couverture
- Créer un écran d'onboarding avec `PageView`
- Ajouter un drawer (menu latéral) en plus de la `BottomNav`

Critères d'évaluation

Critère	Points	Description
BottomNavigationBar	20 pts	4 onglets fonctionnels avec état préservé
Navigation push/pop	25 pts	Détail accessible et retour avec résultat
Passage de données	20 pts	Objet Livre transmis correctement
Routes nommées	15 pts	Configuration et utilisation des routes
Transition animation	10 pts	Transition personnalisée implémentée
UI/UX	10 pts	Cohérence visuelle entre les écrans
TOTAL	100 pts	

TP 4

Formulaires & Validation — Application d'inscription

Durée : 3h · Niveau : Intermédiaire

🎯 Objectifs pédagogiques

- 1 Créer des formulaires complexes avec le widget Form et GlobalKey
- 2 Valider les champs avec des règles métier (regex, longueur, format)
- 3 Utiliser TextFormField avec décoration Material Design 3
- 4 Gérer la soumission de formulaire et l'état de chargement
- 5 Implémenter des champs spéciaux : date, mot de passe, liste déroulante
- 6 Afficher les messages d'erreur de manière ergonomique

📖 Contexte du projet

Scénario

Une plateforme e-learning universitaire doit implémenter un flux complet d'inscription en 3 étapes : informations personnelles, informations académiques et configuration du compte. Chaque étape est un formulaire validé avant de passer à la suivante.

📋 Consignes détaillées

Étape 1 — Structure multi-pages

Utilisez un PageView ou un Stepper widget pour les 3 étapes d'inscription. Implémentez une barre de progression indiquant l'avancement (StepIndicator personnalisé).

Étape 2 — Formulaire Étape 1 : Infos personnelles

```
final _formKey = GlobalKey<FormState>();

Form(
  key: _formKey,
  child: Column(children: [
    TextFormField(
      decoration: InputDecoration(
        labelText: 'Prénom',
        prefixIcon: Icon(Icons.person),
        border: OutlineInputBorder(borderRadius: BorderRadius.circular(12)),
      ),
    ),
```

```

    validator: (value) {
      if (value == null || value.trim().isEmpty)
        return 'Le prénom est obligatoire';
      if (value.trim().length < 2)
        return 'Minimum 2 caractères';
      return null; // Valide
    },
  ),
  // Autres champs...
]),
)

```

Étape 3 — Champs à implémenter

Champ	Type	Validation
Prénom	TextFormField	Obligatoire, ≥ 2 caractères
Nom	TextFormField	Obligatoire, ≥ 2 caractères
Email	TextFormField + regex	Format email valide
Téléphone	TextFormField + regex	Format 10 chiffres
Date de naissance	TextFormField + DatePicker	Majeur (≥ 18 ans)
Pays	DropDownButtonFormField	Sélection obligatoire
Filière	DropDownButtonFormField	Sélection obligatoire
Niveau d'étude	SegmentedButton	L1/L2/L3/M1/M2
Mot de passe	TextFormField + toggle	≥ 8 chars, 1 maj, 1 chiffre
Confirmation MDP	TextFormField	Identique au MDP
CGU	CheckboxFormField	Doit être coché

Étape 4 — Validation au passage d'étape

```

void _nextPage() {
  if (_formKey.currentState!.validate()) {
    _formKey.currentState!.save();
    _pageController.nextPage(
      duration: Duration(milliseconds: 300),
      curve: Curves.easeInOut,
    );
  }
}

```

Étape 5 — Indicateur de force du mot de passe

Ajoutez en temps réel sous le champ mot de passe une barre d'indicateur de force (Faible / Moyen / Fort) avec couleurs (rouge/orange/vert).

Étape 6 — Soumission & état de chargement

À la soumission finale, simulez un appel API (`Future.delayed 2 secondes`). Pendant ce délai, affichez un `CircularProgressIndicator` et désactivez tous les champs.

Arborescence du projet

```
inscription_app/
├── lib/
│   ├── main.dart
│   ├── models/
│   │   └── inscription_data.dart
│   ├── screens/
│   │   ├── inscription_screen.dart
│   │   ├── etape1_screen.dart
│   │   ├── etape2_screen.dart
│   │   ├── etape3_screen.dart
│   │   └── succes_screen.dart
│   ├── widgets/
│   │   ├── step_indicator.dart
│   │   ├── password_strength.dart
│   │   └── custom_text_field.dart
│   ├── utils/
│   │   └── validators.dart
│   └── pubspec.yaml
```

Concepts Flutter utilisés

Form	GlobalKey<FormState>	TextFormField
DropdownButtonFormField	PageView	Stepper
DatePicker	SegmentedButton	CheckboxListTile
validator()	save()	FocusNode
TextInputType	ObscureText	LinearProgressIndicator
CircularProgressIndicator	async/await	Regex Dart

★ Bonus & défis

- Sauvegarder la progression du formulaire si l'utilisateur quitte l'app
- Ajouter un champ 'photo de profil' avec `ImagePicker`
- Validation en temps réel (`onChange`) pour chaque champ
- Internationalisation du formulaire (fr/en)



Critères d'évaluation

Critère	Points	Description
Structure Form/GlobalKey	15 pts	Formulaire structuré correctement
Validations	30 pts	Toutes les règles de validation implémentées
Multi-étapes	20 pts	Navigation entre les 3 étapes avec validation
Champs spéciaux	15 pts	DatePicker, Dropdown, Password toggle
État de chargement	10 pts	Simulation async avec indicateur
Messages d'erreur	10 pts	Ergonomiques et explicites
TOTAL	100 pts	

TP 5

Appels API REST — Application Météo

Durée : 4h · Niveau : Intermédiaire

Objectifs pédagogiques

- 1 Effectuer des requêtes HTTP GET avec le package http
- 2 Parser du JSON en objets Dart avec jsonDecode
- 3 Utiliser FutureBuilder pour gérer les états asynchrones
- 4 Gérer les erreurs réseau et afficher des messages adaptés
- 5 Comprendre le cycle de vie des requêtes (loading/success/error)
- 6 Utiliser les variables d'environnement pour les clés API

Contexte du projet

Scénario

Une agence de voyage souhaite intégrer les données météo en temps réel dans son application. Vous allez créer une application météo connectée à l'API OpenWeatherMap (gratuite) affichant la météo actuelle, les prévisions sur 5 jours et permettant de sauvegarder des villes favorites.

Consignes détaillées

Étape 1 — Configuration

- Créez un compte gratuit sur openweathermap.org et obtenez une clé API
- Ajoutez le package http dans pubspec.yaml : flutter pub add http
- Créez un fichier lib/utils/api_keys.dart (ne jamais committer avec la vraie clé !)

Étape 2 — Service API

Créez lib/services/weather_service.dart :

```
import 'dart:convert';
import 'package:http/http.dart' as http;

class WeatherService {
  static const _baseUrl = 'https://api.openweathermap.org/data/2.5';
```

```

static const _apiKey = 'VOTRE_CLE_API';

Future<MeteoActuelle> getMeteoActuelle(String ville) async {
  final url = Uri.parse('$_baseUrl/weather'
    '?q=$ville&appid=$_apiKey&units=metric&lang=fr');
  final response = await http.get(url);

  if (response.statusCode == 200) {
    final json = jsonDecode(response.body);
    return MeteoActuelle.fromJson(json);
  } else if (response.statusCode == 404) {
    throw Exception('Ville introuvable');
  } else {
    throw Exception('Erreur serveur: ${response.statusCode}');
  }
}

Future<List<PrevisionMeteo>> getPrevisions(String ville) async {
  // Endpoint : /forecast (5 jours, toutes les 3h)
  // À implémenter...
}

```

Étape 3 — Modèles JSON

Créez les classes de données avec constructeurs fromJson :

```

class MeteoActuelle {
  final String ville, pays, description, icone;
  final double temperature, ressenti, tempMin, tempMax;
  final int humidite, vent;

  MeteoActuelle.fromJson(Map<String, dynamic> json)
    : ville = json['name'],
      pays = json['sys']['country'],
      description = json['weather'][0]['description'],
      icone = json['weather'][0]['icon'],
      temperature = (json['main']['temp'] as num).toDouble(),
      ressenti = (json['main']['feels_like'] as num).toDouble(),
      tempMin = (json['main']['temp_min'] as num).toDouble(),
      tempMax = (json['main']['temp_max'] as num).toDouble(),
      humidite = json['main']['humidity'],
      vent = (json['wind']['speed'] as num).toInt();
}

```

Étape 4 — FutureBuilder

```

FutureBuilder<MeteoActuelle>(
  future: _weatherService.getMeteoActuelle(_villeRecherchee),
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return const Center(child: CircularProgressIndicator());
    } else if (snapshot.hasError) {
      return ErrorWidget(message: snapshot.error.toString());
    } else if (snapshot.hasData) {
      return MeteoCard(meteo: snapshot.data!);
    }
    return const SizedBox();
  },
)

```

Étape 5 — Écrans à créer

- Écran principal : météo actuelle de la ville recherchée
- Barre de recherche avec SearchDelegate
- Section prévisions 5 jours (ListView horizontal)
- Détails : humidité, vent, ressenti, pression
- Icônes météo depuis l'URL OpenWeatherMap

Arborescence du projet

```
meteo_app/
├── lib/
│   ├── main.dart
│   ├── models/
│   │   ├── meteo_actuelle.dart
│   │   └── prevision_meteo.dart
│   ├── services/
│   │   └── weather_service.dart
│   ├── screens/
│   │   ├── meteo_screen.dart
│   │   └── recherche_screen.dart
│   ├── widgets/
│   │   ├── meteo_card.dart
│   │   ├── prevision_item.dart
│   │   ├── detail_card.dart
│   │   └── error_widget.dart
│   └── utils/
│       └── api_keys.dart
└── pubspec.yaml
```

Concepts Flutter utilisés

http package	FutureBuilder	jsonDecode
async/await	try/catch	ConnectionState
snapshot.hasData	Image.network	SearchDelegate
RefreshIndicator	CircularProgressIndicator	Gestion erreurs
Modèles fromJson	URI	Headers HTTP

★ Bonus & défis

- Géolocalisation avec geolocator pour la météo automatique
- Cache des résultats pour éviter les appels répétés
- Animations selon la météo (pluie, soleil, neige)
- Widget de bureau (home_widget package)



Critères d'évaluation

Critère	Points	Description
Service HTTP	25 pts	Requêtes GET fonctionnelles avec gestion 404/500
Parsing JSON	20 pts	Modèles fromJson corrects
FutureBuilder	20 pts	3 états gérés (loading/data/error)
Recherche	15 pts	Recherche de ville fonctionnelle
Prévisions 5 jours	10 pts	ListView horizontal des prévisions
UI/UX	10 pts	Interface claire et informative
TOTAL	100 pts	

TP
6

Stockage Local — Application de Notes avec
SharedPreferences & SQLite

Durée : 4h · Niveau : Intermédiaire

 Objectifs pédagogiques

1	Persister des données simples avec SharedPreferences
2	Créer une base de données locale avec sqflite
3	Réaliser les opérations CRUD (Create, Read, Update, Delete)
4	Comprendre la différence entre SharedPreferences et SQLite
5	Utiliser un pattern Repository pour abstraire le stockage
6	Migrer des données et gérer les versions de base de données

 Contexte du projet

Scénario
Une application de prise de notes inspirée de Google Keep / Notion doit fonctionner entièrement hors ligne. Les notes doivent être persistées localement avec SQLite, les préférences utilisateur (thème, tri) avec SharedPreferences. L'application doit survivre aux redémarrages.

Tableau comparatif : SharedPreferences vs SQLite

Critère	SharedPreferences	SQLite (sqflite)
Type de données	Primitifs (bool, int, String, double)	Données structurées complexes
Volume	Quelques Ko	Plusieurs Mo / Go
Requêtes	Clé → valeur uniquement	SQL complet (WHERE, JOIN, ORDER)
Usage typique	Préférences, tokens, flags	Entités métier, listes
Performance	Très rapide	Rapide (indexé)

Persistence	Oui	Oui
-------------	-----	-----

Consignes détaillées

Étape 1 — Installation des packages

```
flutter pub add shared_preferences sqflite path
```

Étape 2 — Modèle Note

```
class Note {
  final int? id;
  String titre, contenu, couleur;
  bool epinglee;
  final DateTime createdAt;
  DateTime updatedAt;

  Note({this.id, required this.titre, required this.contenu,
    this.couleur = '#FFFFFF', this.epinglee = false})
    : createdAt = DateTime.now(), updatedAt = DateTime.now();

  Map<String, dynamic> toMap() => {
    'id': id, 'titre': titre, 'contenu': contenu,
    'couleur': couleur, 'epinglee': epinglee ? 1 : 0,
    'createdAt': createdAt.toIso8601String(),
    'updatedAt': updatedAt.toIso8601String(),
  };

  factory Note.fromMap(Map<String, dynamic> map) => Note(
    id: map['id'], titre: map['titre'], contenu: map['contenu'],
    couleur: map['couleur'], epinglee: map['epinglee'] == 1,
  );
}
```

Étape 3 — DatabaseHelper

```
class DatabaseHelper {
  static Database? _db;
  static const _version = 1;
  static const _dbName = 'notes.db';

  Future<Database> get database async {
    _db ??= await _initDb();
    return _db!;
  }

  Future<Database> _initDb() async {
    final path = join(await getDatabasesPath(), _dbName);
    return openDatabase(path, version: _version, onCreate: _onCreate);
  }

  Future<void> _onCreate(Database db, int version) async {
    await db.execute('''CREATE TABLE notes(
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      titre TEXT NOT NULL,
      contenu TEXT,
      couleur TEXT DEFAULT '#FFFFFF',
      epinglee INTEGER DEFAULT 0,
      createdAt TEXT,
      updatedAt TEXT
```

```

    )''');
  }

  Future<int> insertNote(Note note) async {
    final db = await database;
    return db.insert('notes', note.toMap());
  }

  Future<List<Note>> getAllNotes() async {
    final db = await database;
    final maps = await db.query('notes',
      orderBy: 'epinglee DESC, updatedAt DESC');
    return maps.map((m) => Note.fromMap(m)).toList();
  }

  Future<int> updateNote(Note note) async {
    final db = await database;
    return db.update('notes', note.toMap(),
      where: 'id = ?', whereArgs: [note.id]);
  }

  Future<int> deleteNote(int id) async {
    final db = await database;
    return db.delete('notes', where: 'id = ?', whereArgs: [id]);
  }
}

```

Étape 4 — SharedPreferences pour les préférences

```

class PrefsService {
  static const _themeKey = 'dark_mode';
  static const _triFavoriKey = 'tri_favoris';

  Future<bool> isDarkMode() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getBool(_themeKey) ?? false;
  }

  Future<void> setDarkMode(bool value) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setBool(_themeKey, value);
  }
}

```

Arborescence du projet

```

notes_app/
├── lib/
│   ├── main.dart
│   ├── models/
│   │   └── note.dart
│   ├── services/
│   │   ├── database_helper.dart
│   │   └── prefs_service.dart
│   ├── repositories/
│   │   └── note_repository.dart
│   ├── screens/
│   │   ├── notes_screen.dart
│   │   └── edit_note_screen.dart
│   └── widgets/
│       ├── note_card.dart
│       └── color_picker.dart
└── pubspec.yaml

```

Concepts Flutter utilisés

SharedPreferences	sqflite	openDatabase
CRUD SQL	Singleton pattern	FutureBuilder
GridView.staggeredTiles	GestureDetector	LongPress
PopupMenuButton	Recherche locale	StreamBuilder
path package	Migration DB	Repository pattern

Bonus & défis

- Exporter les notes en fichier texte ou PDF
- Ajouter des images aux notes (stockage local de fichiers)
- Synchronisation Cloud avec Firebase Firestore
- Chiffrement des notes sensibles

Critères d'évaluation

Critère	Points	Description
DatabaseHelper SQLite	25 pts	CRUD complet et correct
Modèle toMap/fromMap	15 pts	Sérialisation et désérialisation
SharedPreferences	15 pts	Persistence des préférences
CRUD fonctionnel	25 pts	Ajout, modification, suppression
Tri & recherche	10 pts	Recherche et tri des notes
UI/UX	10 pts	Interface cohérente avec les données
TOTAL	100 pts	

TP 7

Gestion d'état avancée avec Provider — Application Panier E-commerce

Durée : 4h · Niveau : Intermédiaire avancé

Objectifs pédagogiques

- 1 Comprendre les limites de `setState()` à grande échelle
- 2 Installer et configurer le package provider
- 3 Créer des `ChangeNotifier` pour la gestion d'état globale
- 4 Utiliser `Consumer`, `Provider.of` et `context.watch/read`
- 5 Séparer la logique métier de l'UI (principe de séparation des responsabilités)
- 6 Optimiser les rebuilds avec `Consumer` et `Selector`

Contexte du projet

Scénario

Une startup de commerce électronique souhaite créer une application d'achat en ligne. Le panier doit être accessible depuis n'importe quel écran, les changements doivent se propager automatiquement et la liste de favoris doit persister. Provider est le pattern choisi pour gérer cet état partagé.

Schéma : architecture Provider

Architecture

```

MaterialApp
├── MultiProvider (PanierProvider, ProduitsProvider, FavorisProvider)
│   ├── CatalogueScreen → context.watch<ProduitsProvider>()
│   ├── PanierScreen    → context.watch<PanierProvider>()
│   └── AppBar          → Consumer<PanierProvider> (badge)

```

Consignes détaillées

Étape 1 — Installation

```
flutter pub add provider
```

Étape 2 — PanierProvider

```
class PanierProvider extends ChangeNotifier {
  final List<PanierItem> _items = [];

  List<PanierItem> get items => List.unmodifiable(_items);
  int get nombreArticles => _items.fold(0, (sum, i) => sum + i.quantite);
  double get total => _items.fold(0, (sum, i) => sum + i.sousTotal);

  void ajouterAuPanier(Produit produit) {
    final index = _items.indexWhere((i) => i.produit.id == produit.id);
    if (index >= 0) {
      _items[index] = _items[index].copyWith(
        quantite: _items[index].quantite + 1);
    } else {
      _items.add(PanierItem(produit: produit, quantite: 1));
    }
    notifyListeners(); // Rebuild tous les widgets qui écoutent
  }

  void retirerDuPanier(String produitId) {
    _items.removeWhere((i) => i.produit.id == produitId);
    notifyListeners();
  }

  void viderPanier() {
    _items.clear();
    notifyListeners();
  }
}
```

Étape 3 — Configuration MultiProvider

```
void main() {
  runApp(
    MultiProvider(
      providers: [
        ChangeNotifierProvider(create: (_) => ProduitsProvider()),
        ChangeNotifierProvider(create: (_) => PanierProvider()),
        ChangeNotifierProvider(create: (_) => FavorisProvider()),
      ],
      child: const MonApp(),
    ),
  );
}
```

Étape 4 — Consommer le Provider

```
// Méthode 1 : context.watch (rebuild à chaque changement)
final panier = context.watch<PanierProvider>();
Text('${panier.nombreArticles} articles');

// Méthode 2 : context.read (pas de rebuild, pour actions)
ElevatedButton(
  onPressed: () => context.read<PanierProvider>().ajouterAuPanier(produit),
  child: Text('Ajouter'),
)
```

```
);

// Méthode 3 : Consumer (rebuild ciblé)
Consumer<PanierProvider>(
  builder: (context, panier, child) => Badge(
    label: Text('${panier.nombreArticles}'),
    child: Icon(Icons.shopping_cart),
  ),
);
```

Étape 5 — Écrans à créer

- CatalogueScreen : grille de produits avec bouton 'Ajouter au panier'
- PanierScreen : liste des articles, quantités modifiables, total, checkout
- FavorisScreen : produits favoris avec suppression
- Badge sur l'icône panier dans l'AppBar (nombre d'articles)

Arborescence du projet

```
ecommerce_app/
├── lib/
│   ├── main.dart
│   ├── models/
│   │   ├── produit.dart
│   │   └── panier_item.dart
│   ├── providers/
│   │   ├── panier_provider.dart
│   │   ├── produits_provider.dart
│   │   └── favoris_provider.dart
│   ├── screens/
│   │   ├── main_screen.dart
│   │   ├── catalogue_screen.dart
│   │   ├── detail_produit_screen.dart
│   │   ├── panier_screen.dart
│   │   └── favoris_screen.dart
│   └── widgets/
│       ├── produit_card.dart
│       ├── panier_item_widget.dart
│       └── badge_panier.dart
└── pubspec.yaml
```

Concepts Flutter utilisés

Provider package	ChangeNotifier	notifyListeners()
context.watch<T>()	context.read<T>()	Consumer<T>
Selector<T>	MultiProvider	ChangeNotifierProvider
Panier state	Badge widget	GridView.builder
Quantity stepper	Checkout flow	Provider.of<T>

★ Bonus & défis

- Implémenter un écran de paiement avec Stripe (mode test)
- Persister le panier avec SharedPreferences via ProxyProvider
- Ajouter des animations lors de l'ajout au panier
- Implémenter les codes promo et réductions

Critères d'évaluation

Critère	Points	Description
PanierProvider	30 pts	CRUD panier complet avec notifyListeners
Consommation Provider	25 pts	Utilisation correcte de watch/read/Consumer
MultiProvider	15 pts	3 providers configurés
UI synchronisée	20 pts	Badge et total mis à jour en temps réel
Architecture	10 pts	Séparation logique/UI respectée
TOTAL	100 pts	

TP 8

Firestore & Authentification — Application de Réseau Social

Durée : 4h · Niveau : Avancé

Objectifs pédagogiques

- | | |
|---|---|
| 1 | Intégrer Firebase dans un projet Flutter (FlutterFire CLI) |
| 2 | Implémenter l'authentification Email/Mot de passe avec Firebase Auth |
| 3 | Lire et écrire des données dans Firestore en temps réel |
| 4 | Utiliser StreamBuilder pour les mises à jour en temps réel |
| 5 | Sécuriser les données avec les règles Firestore |
| 6 | Gérer le cycle d'authentification (connexion, déconnexion, persistance) |

Contexte du projet

Scénario

Vous allez créer une application de mini réseau social pour une communauté universitaire. Les étudiants peuvent créer un compte, publier des messages courts (posts), liker les publications d'autres étudiants et voir le fil d'actualité en temps réel.

Consignes détaillées

Étape 1 — Configuration Firebase

1. Créez un projet Firebase sur console.firebase.google.com
2. Activez Authentication > Email/Password
3. Créez une base Firestore en mode test
4. Installez FlutterFire CLI : `dart pub global activate flutterfire_cli`
5. Configurez : `flutterfire configure`
6. Ajoutez les packages : `flutter pub add firebase_core firebase_auth cloud_firestore`

Étape 2 — AuthService

```
class AuthService {  
  final _auth = FirebaseAuth.instance;
```

```

Stream<User?> get userStream => _auth.authStateChanges();
User? get currentUser => _auth.currentUser;

Future<UserCredential> signUp(String email, String password) async {
  return _auth.createUserWithEmailAndPassword(
    email: email, password: password);
}

Future<UserCredential> signIn(String email, String password) async {
  return _auth.signInWithEmailAndPassword(
    email: email, password: password);
}

Future<void> signOut() => _auth.signOut();
}

```

Étape 3 — Firestore : structure des données

Collection	Document	Champs
users	uid	email, nom, avatar, bio, createdAt
posts	autoid	userId, contenu, likes[], createdAt
users/{uid}/following	uid	followingId, followedAt

Étape 4 — Stream temps réel avec StreamBuilder

```

class PostsService {
  final _firestore = FirebaseFirestore.instance;

  Stream<List<Post>> getPostsStream() {
    return _firestore
      .collection('posts')
      .orderBy('createdAt', descending: true)
      .limit(50)
      .snapshots()
      .map((snap) => snap.docs.map((d) => Post.fromFirestore(d)).toList());
  }

  Future<void> publierPost(String contenu, String userId) async {
    await _firestore.collection('posts').add({
      'userId': userId,
      'contenu': contenu,
      'likes': [],
      'createdAt': FieldValue.serverTimestamp(),
    });
  }

  Future<void> toggleLike(String postId, String userId) async {
    final ref = _firestore.collection('posts').doc(postId);
    await _firestore.runTransaction((transaction) async {
      final doc = await transaction.get(ref);
      final likes = List<String>.from(doc['likes'] ?? []);
      likes.contains(userId) ? likes.remove(userId) : likes.add(userId);
      transaction.update(ref, {'likes': likes});
    });
  }
}

```

Étape 5 — Gardien d'authentification

```
// main.dart – Routing basé sur l'état auth
StreamBuilder<User?>(
  stream: AuthService().userStream,
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting)
      return const SplashScreen();
    if (snapshot.hasData)
      return const MainScreen(); // Utilisateur connecté
    return const LoginScreen(); // Non connecté
  },
)
```

Arborescence du projet

```
social_app/
├── lib/
│   ├── main.dart
│   ├── models/
│   │   ├── user_model.dart
│   │   └── post.dart
│   ├── services/
│   │   ├── auth_service.dart
│   │   └── posts_service.dart
│   ├── screens/
│   │   ├── login_screen.dart
│   │   ├── register_screen.dart
│   │   ├── feed_screen.dart
│   │   └── profil_screen.dart
│   ├── widgets/
│   │   ├── post_card.dart
│   │   └── like_button.dart
│   └── firebase_options.dart ← Généré par FlutterFire
└── pubspec.yaml
```

Concepts Flutter utilisés

Firebase Auth	Firestore	StreamBuilder
authStateChanges()	stream.map()	FieldValue.serverTimestamp()
Transaction Firestore	orderBy().limit()	FlutterFire CLI
Règles Firestore	User?	Stream<User?>
Gardien auth	ImagePicker	Upload Storage

★ Bonus & défis

- Authentification Google Sign-In
- Upload de photos de profil avec Firebase Storage
- Notifications push avec Firebase Cloud Messaging (FCM)

- Pagination infinie du fil d'actualité avec Firestore cursors



Critères d'évaluation

Critère	Points	Description
Configuration Firebase	15 pts	Projet configuré et connecté
Authentification	25 pts	Inscription, connexion, déconnexion
Firestore CRUD	25 pts	Lecture et écriture fonctionnelles
StreamBuilder temps réel	20 pts	Mises à jour automatiques du fil
Gardien d'authentification	10 pts	Redirection selon l'état auth
Sécurité	5 pts	Règles Firestore de base
TOTAL	100 pts	

TP 9

UI Avancée & Animations — Application de Fitness

Durée : 4h · Niveau : Avancé

Objectifs pédagogiques

- 1 Créer des animations Flutter avec AnimationController
- 2 Utiliser les animations implicites (AnimatedContainer, AnimatedOpacity...)
- 3 Implémenter des layouts responsifs avec LayoutBuilder et MediaQuery
- 4 Créer des widgets personnalisés avec CustomPainter
- 5 Utiliser les packages d'animation : lottie, animate_do
- 6 Concevoir une UI moderne avec Material Design 3

Contexte du projet

Scénario

Une application de fitness connecté doit offrir une expérience visuelle immersive. Vous allez créer des dashboards animés, des graphiques de progression, des cartes d'exercices avec animations et une interface haute qualité inspirée de Strava et Nike Training Club.

Consignes détaillées

Étape 1 — AnimationController

```
class _WorkoutCardState extends State<WorkoutCard>
    with SingleTickerProviderStateMixin {
  late final AnimationController _controller;
  late final Animation<double> _scaleAnim;
  late final Animation<double> _fadeAnim;

  @override
  void initState() {
    super.initState();
    _controller = AnimationController(
      vsync: this, duration: const Duration(milliseconds: 600));
    _scaleAnim = Tween<double>(begin: 0.8, end: 1.0).animate(
      CurvedAnimation(parent: _controller, curve: Curves.elasticOut));
    _fadeAnim = Tween<double>(begin: 0, end: 1).animate(
      CurvedAnimation(parent: _controller, curve: Curves.easeIn));
    _controller.forward();
  }
}
```

```

@override
void dispose() {
  _controller.dispose();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  return FadeTransition(
    opacity: _fadeAnim,
    child: ScaleTransition(scale: _scaleAnim, child: /* ... */),
  );
}
}

```

Étape 2 — Animations implicites

```

// Compteur animé de calories
TweenAnimationBuilder<double>(
  tween: Tween<double>(begin: 0, end: calories.toDouble()),
  duration: const Duration(seconds: 2),
  curve: Curves.easeOut,
  builder: (_, value, __) {
    return Text('${value.toInt()} kcal',
      style: TextStyle(fontSize: 48, fontWeight: FontWeight.bold));
  },
);

```

Étape 3 — CustomPainter : graphique de progression

```

class ProgressPainter extends CustomPainter {
  final double progress; // 0.0 à 1.0
  final Color color;
  ProgressPainter({required this.progress, required this.color});

  @override
  void paint(Canvas canvas, Size size) {
    final paint = Paint()
      ..strokeWidth = 12
      ..strokeCap = StrokeCap.round
      ..style = PaintingStyle.stroke;

    // Arc de fond (gris)
    paint.color = Colors.grey.shade200;
    canvas.drawArc(Rect.fromLTWH(0, 0, size.width, size.height),
      -pi / 2, 2 * pi, false, paint);

    // Arc de progression (coloré)
    paint.color = color;
    canvas.drawArc(Rect.fromLTWH(0, 0, size.width, size.height),
      -pi / 2, 2 * pi * progress, false, paint);
  }

  @override
  bool shouldRepaint(ProgressPainter old) => old.progress != progress;
}

```

Étape 4 — Layout responsive

```

LayoutBuilder(
  builder: (context, constraints) {

```

```
final isWide = constraints.maxWidth > 600;
return isWide
  ? Row(children: [SideMenu(), Expanded(child: Content())])
  : Column(children: [Content()]);
},
);
```

Étape 5 — Écrans à créer

- Dashboard : anneau de calories animé, stats du jour, objectifs
- Liste d'exercices : cards avec animation d'entrée séquentielle
- Timer d'entraînement : countdown animé avec vibration
- Historique : graphique de progression avec CustomPainter

Arborescence du projet

```
fitness_app/
├── lib/
│   ├── main.dart
│   ├── models/
│   │   ├── exercice.dart
│   │   └── seance.dart
│   ├── screens/
│   │   ├── dashboard_screen.dart
│   │   ├── exercices_screen.dart
│   │   ├── timer_screen.dart
│   │   └── historique_screen.dart
│   ├── widgets/
│   │   ├── calories_ring.dart
│   │   ├── progressPainter.dart
│   │   ├── exercice_card.dart
│   │   └── animated_counter.dart
│   ├── utils/
│   │   └── app_theme.dart
└── pubspec.yaml
```

Concepts Flutter utilisés

AnimationController	Tween	CurvedAnimation
AnimatedContainer	AnimatedOpacity	TweenAnimationBuilder
CustomPainter	Canvas	LayoutBuilder
MediaQuery	ScaleTransition	FadeTransition
SlideTransition	SingleTickerProviderStateMixin	Lottie
Vibration	Timer	CircularProgressIndicator

★ Bonus & défis

- Intégrer des animations Lottie pour les icônes d'exercices

- Ajouter des confettis à la fin d'une séance (confetti package)
- Créer un graphique de progression en barres avec CustomPainter
- Implémenter des thèmes dynamiques (5 couleurs au choix)



Critères d'évaluation

Critère	Points	Description
AnimationController	25 pts	Animation explicite fonctionnelle
Animations implicites	20 pts	Au moins 3 widgets animés implicitement
CustomPainter	20 pts	Graphique ou anneau de progression
Responsive	15 pts	Adaptation tablette/mobile
Qualité UI	10 pts	Design moderne et cohérent
Performance	10 pts	dispose() appelé, pas de fuites mémoire
TOTAL	100 pts	

TP 10

Architecture Propre & Tests — Application Actualités

Durée : 4h · Niveau : Avancé

Objectifs pédagogiques

1	Appliquer l'architecture Clean Architecture en Flutter
2	Utiliser Riverpod ou BLoC comme solution de gestion d'état avancée
3	Écrire des tests unitaires avec le package test
4	Écrire des tests de widgets avec flutter_test
5	Comprendre le principe de l'injection de dépendances
6	Documenter son code avec des commentaires dartdoc

Contexte du projet

Scénario

Une agence de presse numérique veut refondre son application mobile avec une architecture maintenable et testable à long terme. Vous allez créer une application d'actualités en appliquant Clean Architecture, avec des tests unitaires pour la couche métier et des tests de widgets pour l'UI.

Schéma : Clean Architecture Flutter

Couche	Responsabilité	Exemples
Présentation	UI + gestion d'état	Screens, Widgets, Providers/BLoCs
Domaine	Logique métier pure	Entities, UseCases, Repository interfaces
Data	Accès aux données	Models, API, DB, Repository implementations

Consignes détaillées

Étape 1 — Structure Clean Architecture

```

news_app/lib/
├── core/
│   ├── error/
│   │   └── failures.dart
│   ├── usecases/
│   │   └── usecase.dart
│   └── network/
│       └── network_info.dart
├── features/
│   └── articles/
│       ├── domain/
│       │   ├── entities/
│       │   │   └── article.dart          ← Entité pure (pas de JSON)
│       │   ├── repositories/
│       │   │   └── article_repository.dart ← Interface abstraite
│       │   └── usecases/
│       │       ├── get_top_articles.dart
│       │       └── search_articles.dart
│       ├── data/
│       │   ├── models/
│       │   │   └── article_model.dart    ← Article + fromJson
│       │   ├── datasources/
│       │   │   └── news_remote_datasource.dart
│       │   └── repositories/
│       │       └── article_repository_impl.dart
│       └── presentation/
│           ├── pages/
│           │   └── news_page.dart
│           ├── widgets/
│           │   └── article_card.dart
│           └── providers/ (ou blocs/)
│               └── articles_provider.dart
└── main.dart
  
```

Étape 2 — Entité vs Modèle

```

// DOMAINE – Entité pure, pas de dépendances JSON
class Article {
  final String id, titre, description, url, imageUrl, source;
  final DateTime publishedAt;
  const Article({required this.id, required this.titre, ...});
}

// DATA – Modèle avec logique JSON
class ArticleModel extends Article {
  const ArticleModel({required super.id, required super.titre, ...});

  factory ArticleModel.fromJson(Map<String, dynamic> json) {
    return ArticleModel(
      id: json['url'] ?? '',
      titre: json['title'] ?? 'Sans titre',
      description: json['description'] ?? '',
      url: json['url'] ?? '',
      imageUrl: json['urlToImage'] ?? '',
      source: json['source']['name'] ?? '',
      publishedAt: DateTime.parse(json['publishedAt']),
    );
  }
}
  
```

Étape 3 — UseCase

```
class GetTopArticles implements UseCase<List<Article>, String> {
  final ArticleRepository repository;
  GetTopArticles(this.repository);

  @override
  Future<Either<Failure, List<Article>>> call(String categorie) {
    return repository.getTopArticles(categorie);
  }
}
```

Étape 4 — Tests unitaires

```
// test/features/articles/domain/get_top_articles_test.dart
import 'package:mockito/mockito.dart';
import 'package:test/test.dart';

class MockArticleRepository extends Mock implements ArticleRepository {}

void main() {
  late GetTopArticles useCase;
  late MockArticleRepository mockRepository;

  setUp(() {
    mockRepository = MockArticleRepository();
    useCase = GetTopArticles(mockRepository);
  });

  test('devrait retourner une liste d articles', () async {
    // Arrange
    final articles = [Article(id:'1', titre:'Test', ...)];
    when(mockRepository.getTopArticles(any))
      .thenAnswer((_) async => Right(articles));

    // Act
    final result = await useCase('tech');

    // Assert
    expect(result, Right(articles));
    verify(mockRepository.getTopArticles('tech'));
  });
}
```

Étape 5 — Tests de widgets

```
// test/features/articles/presentation/article_card_test.dart
void main() {
  testWidgets('ArticleCard affiche le titre', (tester) async {
    final article = Article(id:'1', titre:'Flutter Test', ...);

    await tester.pumpWidget(
      MaterialApp(home: ArticleCard(article: article)),
    );

    expect(find.text('Flutter Test'), findsOneWidget);
    expect(find.byType(Image), findsOneWidget);
  });
}
```

Concepts Flutter utilisés

Clean Architecture	UseCase	Repository pattern
Either<Failure,T>	Riverpod / BLoC	Mockito
flutter_test	testWidgets()	pumpWidget()
find.text()	verify()	Injection dépendances
Dart doc	Either (dartz)	Abstract classes

★ Bonus & défis

- Implémenter avec flutter_riverpod 2.x complet
- Atteindre 80% de couverture de tests (flutter test --coverage)
- Ajouter des tests d'intégration avec integration_test
- Générer la documentation avec dart doc

Critères d'évaluation

Critère	Points	Description
Structure Clean Architecture	25 pts	3 couches correctement séparées
UseCase implémentés	20 pts	Au moins 3 use cases
Tests unitaires	25 pts	5+ tests passants sur les use cases
Tests de widgets	15 pts	3+ tests de widgets
Documentation	10 pts	Commentaires dartdoc sur les classes publiques
Qualité code	5 pts	flutter analyze sans warnings
TOTAL	100 pts	



Mini-Projet Final Intégrateur

Durée : 2 semaines · Travail en binôme ou individuel · Évaluation orale + code

Sujet : Application EduHub — Plateforme d'apprentissage mobile

EduHub est une application mobile complète de plateforme d'apprentissage en ligne. Ce projet intègre TOUS les concepts vus dans les 10 TP précédents.

Fonctionnalités obligatoires

Fonctionnalité	TP source	Technologie
Inscription / Connexion	TP 4 + TP 8	Firebase Auth + Formulaires
Catalogue de cours	TP 3 + TP 5	Navigation + API REST
Lecteur de leçons	TP 3	Navigation + vidéo
Quiz interactif	TP 2 + TP 7	ListView + Provider/Riverpod
Progression & statistiques	TP 9	Animations + CustomPainter
Notes de cours	TP 6	SQLite local
Profil & paramètres	TP 1 + TP 4	Forms + SharedPreferences
Architecture	TP 10	Clean Architecture + Tests

Architecture attendue

```
eduhub/lib/  
├── core/  
│   ├── error/  
│   ├── network/  
│   └── theme/  
├── features/  
│   ├── auth/  
│   ├── cours/  
│   ├── quiz/  
│   ├── profil/  
│   └── notes/  
└── main.dart
```

Grille d'évaluation finale (100 points)

Critère	Points	Description
Fonctionnalités	30 pts	Toutes les fonctionnalités requises opérationnelles
Architecture	20 pts	Clean Architecture respectée
Tests	15 pts	Tests unitaires et widgets
Firebase	15 pts	Auth et Firestore intégrés
UI/UX	10 pts	Design professionnel et cohérent
Code quality	10 pts	Lisibilité, nommage, commentaires
TOTAL	100 pts	



Idée d'Examen Pratique Flutter

Durée : 3 heures · Ressources autorisées : documentation Flutter officielle uniquement · Pas de collaboration

Sujet d'examen suggéré

Application de gestion de contacts L'étudiant doit créer en 3h une application de gestion de contacts avec : liste de contacts (ListView), ajout/modification/suppression, recherche, stockage SQLite et navigation. Les données doivent persister entre les sessions.

Barème de l'examen

Critère	Points	Description
Projet crée & exécuté	10 pts	L'app compile sans erreur
Modèle Contact	10 pts	Classe avec tous les champs requis
Liste avec ListView.builder	15 pts	Contacts affichés correctement
Ajout d'un contact (Form)	20 pts	Formulaire avec validation
Suppression (Dismissible)	15 pts	Balayage pour supprimer
Recherche	10 pts	Filtrage en temps réel
SQLite persistance	15 pts	Données conservées au redémarrage
UI/UX	5 pts	Interface propre et utilisable
TOTAL	100 pts	



Pour aller plus loin

Ressources officielles

- flutter.dev/docs — Documentation officielle Flutter
- dart.dev/guides — Guide du langage Dart
- pub.dev — Catalogue de packages Flutter
- flutter.dev/community — Communauté et Discord

Packages incontournables à maîtriser

Package	Catégorie	Utilité
riverpod	État	Gestion d'état moderne et testable
go_router	Navigation	Navigation déclarative type-safe
dio	HTTP	Client HTTP avancé avec intercepteurs
freezed	Modèles	Classes immuables et pattern matching
hive	Stockage	Base de données NoSQL ultra-rapide
cached_network_image	Images	Cache d'images réseau
flutter_localizations	i18n	Internationalisation
flutter_bloc	État	Pattern BLoC éprouvé

Roadmap de progression

1. Maîtriser Riverpod 2.x pour la gestion d'état
2. Apprendre les tests d'intégration (`integration_test`)
3. Explorer Flutter Web et Flutter Desktop
4. Étudier les principes SOLID appliqués à Flutter
5. Contribuer à des projets open source Flutter
6. Publier une application sur le Play Store / App Store



Conseil final

La meilleure façon d'apprendre Flutter est de construire de vraies applications. Reproduisez des apps que vous utilisez quotidiennement : un clone simplifié de WhatsApp, Instagram, ou Duolingo. Chaque obstacle rencontré est une opportunité d'apprentissage.